provides two modes of execution during development:

- *Host/Dev mode:* Written Java code is executed directly in Web browsers through embedded DOM elements. Java byte-code interacts directly with the JVM for rendering UI elements, without converting it into JavaScript. This provides an easy way for the developer to debug applications using IDEs.
- *Web/Prod mode:* The Java-to-JavaScript compiler is the backbone of the GWT framework. Generated JavaScript will run and create UI elements, and this JavaScript runs well in all browsers.

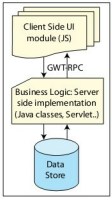

Figure 1: GWT client architecture

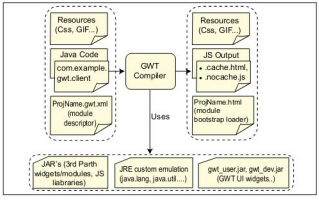

Figure 2: An overview of the GWT application

The core GWT functionality and UI elements are provided by *gwt-user.jar* and *gwt-dev.jar*. The overall view of AJAX applications developed using GWT framework is shown in Figure 2.

UI elements can interact with server-side business logic through any approach such as the GWT-RPC mechanism, traditional servlets, or by building custom HTTP requests to retrieve information from the server using GWT's HTTP client class in the *com.google.gwt.http.client* package. If the server-side implementation is written in Java, then GWT-RPC is the best choice for the client-server communication. GWT RPC is a mechanism for passing serialised Java objects to and from a server over standard HTTP. The server-side implementation will not be converted into JavaScript, so it can contain any Java classes or external libraries.

Hidden features of GWT

The points mentioned above are some of the well-known important features of GWT. There are many less-known features, which help developers build applications with better performance and quality. These features will also improve modularity while developing huge Web applications. In this article, I will highlight these not-so-common, yet important features of GWT, and provide some insights into them.

Client-side application unit testing

Testing applications and understanding the internals, reveals bugs in the early stages of development and helps developers deliver quality products. GWT provides testing for the client-side part of the application, to simulate the actions after firing certain events. It can also be used to test the performance of the asynchronous function calls, to get the server response or execution time. *JUnit* is the most commonly used open source unit testing framework for Java applications. The GWT framework has extended *JUnit* and customised it for testing both Web and hosted modes of GWT applications. To implement a test case, a class has to extend *GWTTestCase* class, override and implement the *getModuleName()* function, which gives the entry point (class name) of the application.

```
public GWTExampleTestCase extends GWTTestCase{
   public String getModuleName(){
      return "com.example.gwt.test.client.TestCase";
   }
   public void testCase1(){
      //implement test case
      assert(true);
   }
   public void rpcTestCase2(){
      //implement rpc object creation & function calls
      onSuccess(){
         finishTest()
         // Successful execution, if flow reaches this function
// before maximum time expires
      }
      onFailure(){
         assert(false);
      }
      delayTestFinish(maximumAllowedTimeMilliSecs);
      // For testing execution time of server calls
   }
}
```

Multiple test case classes can be executed together by writing a test suite (a class that extends the *GWTTestSuite* class, and defines all test case classes together). GWT provides a batch script, *junitCreator,* which creates a skeleton GWT test unit class, along with Ant scripts for testing both Dev and Web modes.

Client-side logging

Since we can't use external logging libraries like *log4j* on the client side, GWT provides a separate module for this, which emulates *java.util.logging*, so the method of configuring and accessing it is the same as traditional logging in Java applications. The GWT log module has to be inherited in the project module descriptor file: